0. 環境與 Python 特性 (Lec1/2)

- 虛擬環境 venv: 隔離各專案的套件相依, 避免版本衝突, 不影響系統 Python (`python -m venv .venv`, activate 後再 `pip install`)。
- 直譯式 (REPL, 免編譯)、動態型別 (同一變數可換型別)、大小寫敏感 (`Print()` NameError, `If` SyntaxError)、Batteries included (標準庫完整)。
- Zen: Explicit > implicit, Simple > complex, Flat > nested, Sparse > dense, Readability counts, 一種明顯做法。

1. 縮排 / PEP 8 / 命名

- 縮排定義區塊, 4 空白, 同層一致, 不可混用 tab 與 space (IndentationError)。
- 常見錯誤: `if x:` 後未縮排 → `expected an indented block`; 多縮排 → `unexpected indent`; 同區塊縮排不一致; 缺冒號 → SyntaxError; `def` 內忘了縮排; 巢狀區塊每層 +4。
- PEP 8 = 官方風格指南 (Guido 等, 2001): 一致性/可讀性/協作/業界標準/工具支援。程式每行 ≤79 字元, 註解與 docstring ≤72; 運算子兩側空白 `a = b + c`; keyword arg 的 `=` 兩側不加空白 `f(x, k=3)`; 切片 `a[:3]`; import 分組 (標準庫 / 第三方 / 本地) 以空行隔開; 註解 `#`, `"""doc"""`, `# TODO`, `# FIXME`。
- 命名: 變數/函式 `snake_case`; 類別 `PascalCase`; 常數 `UPPER_CASE`; `_x` 內部使用慣例; `userName`(camelCase)、`Calculate_Total` 皆不符。
- 變數規則: 字母或 _ 開頭、可含字母/數字/底線、區分大小寫、不可為關鍵字 (共 35 個: `class def lambda None True False and or not in is if elif else for while break continue pass return try except finally raise with as import from global nonlocal del assert yield async await`)。`2name user-name class` 皆非法。
- 工具: pylint (完整分析), flake8 (風格檢查), black (自動排版), autopep8 (自動修正); PEP 8 是準則非鐵律 (可讀性/相容舊碼可例外, 學習時嚴格遵守)。

2. 型別與運算子

- 基本型別: `int float complex(3+4j) str bool`; `type(x)`; `type(x)==int`; `isinstance(x,(int,float))` (可多型別 tuple)。
- 算術: `/` 真除法 (恆為 float) `//` 取整 (floor) `%` 餘數 `**` 次方; `-7//2=-4`, `-7%2=1`; 比較 `== != < > <= >=` 可串接 `0<=x<=100`; 字串依字典序 `"apple"<"banana"`。
- 邏輯 `and or not`; 短路: `(x!=0) and (10/x>1)` 不會除零; `or` 左邊為真就不求右邊。
- 位元: `& | ^ ~ << >>`; `10=1010,4=0100`: `&`=0, `|`=14, `^`=14, `~10=-11`, `10<<1=20`, `10>>1=5`。
- 轉換: 隱式 int+float → float; 顯式 `int("123")` `float("1.5")` `bool(0)` `str(5)`; `int("3.14")` ValueError; `int(3.9)=3` (往 0 截斷)。Falsy: `0 0.0 "" [] () {} set() None`; `bool("False")=True`。
- 四捨五入 (銀行家): `f"{2.5:.0f}"=2`, `f"{3.5:.0f}"=4`。

3. 流程控制與輸入模板

- `if / elif / else`: 由上而下, 第一個成立就停。三元: `x = a if cond else b`。
- 風格: 巢狀 (C-style) → guard clause 先擋非法再線性判斷 `if not 0<=s<=100: return "Invalid"` → 邊界資料化 `[(90,"A"),(80,"B")]` 迴圈掃描 (皆 O(n), 要量測 `time.perf_counter()`, 可讀性優先)。
- `for x in iterable` (list/str/dict.items/range); `range(stop)`, `range(a,b)`, `range(a,b,step)` (stop 不含); 倒數 `range(10,0,-1)`。`while cond:` 記得更新條件。
- `break` 跳出整個迴圈; `continue` 跳下一圈; `pass` 佔位 (空函式/類別/分支)。巢狀迴圈: 九九乘法、三角形 `print("*"*(i+1))`。優化: 不變運算移出迴圈。

```
n = int(input())                          # input() 回傳 str
line = input()                            # 整行(保留空白, 勿 strip)
a = list(map(int, input().split()))       # 一行多個數
while True:                               # 哨兵: END / 0 結束
    s = input()
    if s == "END": break
nums = []
while True:                               # 讀到 EOF 為止
    try: x = int(input())
    except EOFError: break
    nums.append(x)
a, b = b, a                               # 交換
arr[j], arr[j+1] = arr[j+1], arr[j]
```

4. 四大集合總覽 (Lec3)

	list `[]`	tuple `()`	set `{}`	dict `{k:v}`
有序	Y	Y	N	Y (3.7+)
可變	Y	N	Y	Y
可重複	Y	Y	N	key 唯一
索引	位置	位置	無	用 key
`x in`	O(n)	O(n)	O(1)	O(1) (key)
取值	O(1)	O(1)	–	O(1)
新增	尾 O(1)/中 O(n)	–	O(1)	O(1)
刪除	O(n)	–	O(1)	O(1)

(set/dict 的 O(1) 為平均, 最差 O(n))

- `{}` 是空 dict, 空 set 用 `set()`; 單元素 tuple 要逗號 `(42,)`, `(42)` 只是 int; `x, y = 10, 20` 不加括號也是 tuple。
- 轉換: `tuple(L)` `set(L)` `list(s)` `dict([("a",1)])` `dict(zip(k,v))` `list("abc")`; `set(L)` 去重但順序不保證。
- 選用: list= 會變動的序列; tuple= 固定記錄/座標/可當 dict key; set= 唯一 + 快速查找; dict=key → value。

5. 記憶體模型 / 複製 / 傳參數

- 變數 = 貼在物件上的標籤, 不是盒子。`b = a` 兩標籤同一物件 (`a is b` True): `b.append(4)` 則 `a` 也變。
- 不可變: int float str tuple frozenset bool; 可變: list set dict。`s += "x"` 建新物件 (id 變); `L.append()` 同一物件 (id 不變)。
- 複製: `a.copy()`, `list(a)`, `a[:]` 為淺拷貝 (巢狀內層仍共用); `copy.deepcopy(a)` 完全獨立。
- 傳參 = call by object reference: 函式內就地修改(`lst.append`) 呼叫端看得到; 重新綁定(`n=0`, `s=s+"x"`) 呼叫端不變。
- 預設參數只在 def 時求值一次: `def add(x, acc=[])` → `add(1)=[1]`, `add(2)=[1,2]`; 修: `acc=None` 再 `if acc is None: acc=[]`。
- 共用陷阱: `[[0]*3]*3` 三列同一個 list → `[[0]*3 for _ in range(3)]`; `dict.fromkeys(keys, [])` 共用同一 list → `{k: [] for k in keys}`。
- hashable: 只有不可變物件可當 dict key / set 元素; `{[1,2]:1}` TypeError, `{(1,2):1}` OK。list= 動態陣列; set/dict= 雜湊表 (相等的 key 必須雜湊相同, 故 key 不能變)。
- 字串組合用 `"".join(parts)`, 迴圈內 `+=` 是 $O(n^2)$。

6. List

```
a = [0,1,2,3,4,5,6,7,8,9]
a[1:4]; a[:5]; a[5:]; a[::2]; a[::-1]; a[-3:]
a[1:4] = [10,20,30]    # 取代(長度可不同)
a[2:2] = [100]         # 插入
del a[1:3]             # 刪切片
[0]*5;  list(range(1,6));  [x**2 for x in range(5)]
```

- 索引從 0, 負索引 -1 為最後; 單一索引越界 IndexError, 切片不會報錯。
- `append(x)` 尾端; `insert(i,x)`; `extend(it)` 多個; `+` 產生新 list; `remove(v)` 刪第一個 (沒有 ValueError); `pop()` / `pop(i)` 刪並回傳; `clear()`; `del a[i]`; `index(v)` (沒有 ValueError); `count(v)`; `in`。
- `a.sort()` 就地排序回傳 None; `sorted(a)` 回傳新 list; `reverse=True`; `key=lambda x: x[1]`; `a.reverse()`; `len min max sum`。
- 推導式: `[expr for x in it if cond]`; 有 else 的三元要放前面 `["e" if x%2==0 else "o" for x in r]`; 巢狀 `[(i,j) for i in r1 for j in r2]`; 攤平 `[n for row in m for n in row]`。比 for+append 簡潔且通常較快, 但不能完全取代 for (需 break、複雜邏輯、副作用時)。

7. Tuple / namedtuple

- 有序、可重複、不可變 (`t[0]=1` TypeError)、hashable (可當 key); 方法只有 `count` `index`; 支援索引/切片/`len`/`in`。「修改」= 轉 list 再轉回, 或 `(10,)+t[1:]`。
- 解包: `x, y = pt`; 交換 `a, b = b, a`; `first, *mid, last = nums` (`mid` 是 list); `_, _, third, *_ = nums`; 函式 `return a, b` 實為回傳 tuple, 可 `mn, mx, avg = stats(L)`。
- `P = namedtuple("P", ["name","age"])` 或 `"x y"`; `p.name` 或 `p[0]`; 可解包、不可變、可當 key; `p._asdict()` `p._replace(age=26)` `P._fields`。

8. Set

- `add(x)`; `update(it)` (`update("ab")` 加入每個字元); `remove(x)` 不存在 KeyError; `discard(x)` 不報錯; `pop()` 任意一個; `clear()`; `in` O(1)。
- 運算 (皆有方法版): 聯集 `A|B union`; 交集 `A&B intersection`; 差集 `A-B difference`; 對稱差 `A^B`; 子集 `<= issubset`; 真子集 `<`; 超集 `>= issuperset`; `isdisjoint`。`&` 優先於 `|`, 不確定就加括號。
- 推導式 `{x**2 for x in r}`; 去重保序: `list(dict.fromkeys(L))` 或 seen 集合; `frozenset(...)` 不可變、可當 key (`add` → AttributeError)。大量成員查詢用 set 不用 list。

Lab3 / HW3 模板

```
# 3-1 共同好友 (set 交集)
n,m = map(int,input().split())
A = set(map(int,input().split())); B = set(map(int,input().
split()))
c = sorted(A & B); print(len(c)); print(*c)
# 3-2 火車調度: C 站當 stack (UVa514: 單獨 0 結束該組, N=0 結
束全部)
def ok(n, tg):
    st=[]; nx=1
    for t in tg:
        while (not st or st[-1]!=t) and nx<=n:
            st.append(nx); nx+=1
        if not st or st[-1]!=t: return False
        st.pop()
    return True
# HW3-1 最高頻字元
from collections import Counter
print(Counter(s).most_common(1)[0][0])
# HW3-2 干擾對 (tuple + set): 距離平方<=D 且 power 不同
if d2 <= D and a[3] != b[3]:
    pairs.add((min(a,b), max(a,b)))     # 小的在前去重
for p,q in sorted(pairs): print(f"{p} <-> {q}")
```

9. Dict (Lec4)

- 建立: `{"a":1}` `dict(a=1)` `dict(zip(k,v))` `{x:x**2 for x in r}` `dict.fromkeys(keys, 0)`。key 唯一且 hashable; 保持插入順序 (3.7+)。
- `d[k]` 缺 key KeyError; `d.get(k, default)` 不報錯; `d[k]=v` 新增/更新; `d.update({...})`; `del d[k]`; `d.pop(k[,default])`; `popitem()` 移除最後一組; `keys()` `values()` `items()`。`k in d` O(1) (查 key); `v in d.values()` O(n)。
- `{1:"a", 1.0:"b", True:"c"}` → `{1: 'c'}`, len=1 (1==1.0==True 雜湊相同 → 同一 key: 保留第一個 key 物件, 最後的 value)。
- 迴圈中增刪 key → RuntimeError; 修: `for k in list(d):`。`for k,v in d.items():` 取 key 與 value。
- 合併 (後者覆蓋前者): `{**a, **b}`; `a | b` (3.9+); `a.update(b)`。反轉 `{v:k for k,v in d.items()}` (重複的 value 會被覆蓋)。
- Counter: `Counter(words)`, `.most_common(k)`, 缺 key 為 0; 手寫 `d[w]=d.get(w,0)+1`。分組: `d.setdefault(k,[]).append(v)`; `defaultdict(list)`; `defaultdict(int)` (缺 key 自動建預設值)。
- 查表取代長 if/elif: `m.get(x, default)`。dispatch table: `ops={"+":operator.add,...}`; `f=ops.get(op)`; `f(a,b)` (類似 C 的 function pointer 表)。
- 巢狀 dict (JSON 形): `db["001"]["courses"][0]`; 層級可能缺用 `db.get("x",{}).get("name","N/A")`。`copy()` 淺拷貝, 巢狀要 `deepcopy`。
- 用 id 建索引: `by_id={s["id"]:s for s in L}` 之後每次查詢 O(1) (list 逐筆找是 O(n))。

10. String

- 不可變 Unicode 序列: `s[0]="H"` TypeError, 只能 `"H"+s[1:]`; 沒有 char 型別 (`type(s[0])` 是 str); `len` O(1) 算字元數 (C 的 strlen 算 bytes)。
- UTF-8: ASCII 1 byte, 常用中文 3 bytes; `len("台灣")=2`, `len("台灣".encode("utf-8"))=6`; `.encode()` / `.decode()`; `ord("台")=0x53f0`, `chr()`。
- 切片 `s[a:b:c]`, `s[::-1]` 反轉, `s[-1]`; 單一索引越界 IndexError。取副檔名 `fn[fn.rfind(".")+1:]`。
- 大小寫 `upper lower capitalize title swapcase casefold`。搜尋 `find` (沒有 → -1) 對 `index` (ValueError); `rfind count startswith endswith in`。
- `strip/lstrip/rstrip(chars)` 的參數是字元集合, 不是前綴: `"https://shop.example.com/".strip("https://")` → `'op.example.com'`; 去前後綴用 `removeprefix` / `removesuffix` (3.9+)。`replace(old,new[,count])`。
- `split()` 吃連續空白; `'a  b c'.split(" ")` → `['a','','b','c']`; `split(sep,1)` 切一次; `splitlines()`; `"-".join(it)` 元素必須是 str (`",".join(map(str,nums))`); `partition("@")` 恆回 3 段 (前, sep, 後)。
- `isalpha isdigit isdecimal isnumeric isalnum isupper islower isspace` 都不認"-5" "3.14", 且 "²" "四" 與 `int()` 不一致; 驗證數字用 `try: int(s) / except ValueError`。`any(c.isupper() for c in pw)`。
- 對齊 `center ljust rjust zfill` (`"7".zfill(3)="007"`)。

11. 格式化 (f-string)

- 三種: `"%s %.1f" % (a,b)` (C printf 風格), `"{} {:.1f}".format(a,b)`, `f"{a} {b:.1f}"` (3.6+, 最快最簡潔, 可放運算式)。格式: `{值:填充 對齊 寬度 ,分組 .精度 型別}`, 依序且皆可省略。對齊 `<` 左 `>` 右 `^` 中 (str 預設左, 數字預設右)。

```
f"{1234.5:*>12,.2f}"    # ****1,234.50
f"{42:05d}"             # 00042
f"{0.856:.1%}"          # 85.6%
f"{1234567.891:,.2f}"   # 1,234,567.89
f"{1234567.891:.3e}"    # 1.235e+06
```

```
f"{255:b} {255:o} {255:x} {255:#x}"  # 11111111 377 ff 0xff
f"{7:+d}"                # +7
f"{x=}"  f"{x*2 = }"     # x=42  x*2 = 84  (3.8+)
f"{name!r}"              # 'Ann'   (!r = repr)
f"[{x:>{w}}]"            # 巢狀寬度
f"{d:%Y/%m/%d}"          # datetime 格式
f"{'pass' if x>=60 else 'fail'}"
print(*nums, sep=", "); print(x, end="")
f"{'Item':<8}{'Price':>8}"   # 對齊表格
"#"*filled + "-"*(w-filled)  # 文字進度條
```

12. 函式 (Lec5)

- `def f(a, b=1, *args, **kwargs)`: 順序: 一般 → 預設 → `*args` (收成 tuple) → `**kwargs` (收成 dict)。呼叫: positional 依順序, keyword 不論順序, 但 positional 必須在 keyword 前 (`f(name="D", 35)` SyntaxError)。
- 呼叫端解包 (方向相反): `f(*lst) f(**dct) print(*nums, sep=",")`。
- 無 `return` → 回傳 `None`; `return a, b` 回傳 tuple; early return / guard clause; docstring + `help(f)`。預設參數陷阱見 §5。
- LEGB: Local → Enclosing → Global → Built-in。函式內賦值就是建立 local; 先讀後寫 global 變數 → UnboundLocalError, 要 `global x`; 修改外層函式變數用 `nonlocal x`。

13. 模組與套件

- 模組 = 一個 `.py`: `import m` / `from m import f,g` / `import m as alias` / `from m import *` (不建議: 污染命名空間)。import 時會執行模組頂層程式。
- `if __name__ == "__main__"`: 直接執行時 `__name__=="__main__"`, 被 import 時為模組名 → 放測試/主程式, import 時不會跑。
- 套件 = 含 `__init__.py` 的資料夾; `__init__.py` 可 re-export (`from .math_tools.basic import add`, `.` 為相對 import), `__all__` 控制 `import *`, 可放 `__version__`。`from pkg.sub import mod as m`。
- 標準庫: `math` (`pi sqrt ceil floor factorial gcd`), `random` (`randint(a,b)` 含兩端, `choice shuffle random`), `datetime` (`now() date.today() timedelta(days=1)`), `os` (`getcwd path`), `sys collections itertools functools`。第三方 `pip install`。

14. 一等函式 / 閉包 / 高階函式

- 函式是物件: `g = f` (不加括號 = 複製參照; `g = f()` 是呼叫後存回傳值), 可放進 list/dict、當引數、當回傳值 (`ops["add"](5,3)`)。
- Closure 閉包: 內層函式捕獲外層變數, 外層返回後狀態仍存活 (`make_counter` + `nonlocal`); 是 decorator 的基礎。
- Late binding: `[lambda: i for i in range(3)]` 呼叫結果皆 `[2,2,2]` (捕獲變數不是值); 修 `lambda i=i: i` → `[0,1,2]`。
- 高階函式 = 吃函式或回傳函式; 工廠 `create_multiplier(3)` 回傳 closure。型別註記 `def f(x: int) -> int, Callable[[int], int]` 僅文件用途, 執行時不檢查。
- lambda: 匿名、僅能單一運算式 (不能有陳述式/多分支), `sorted(L, key=lambda s: s["age"])`。適用: 作為 map/filter/sort 的簡單引數。

```
list(map(int, ["1","2"]))                  # map -> iterator, 要 list()
list(map(lambda x,y: x+y, [1,2], [3,4]))   # [4,6]
list(filter(lambda x: x%2==0, L))
list(filter(None, [1,None,"",0,2]))        # [1,2] 去 falsy
from functools import reduce
reduce(lambda a,b: a+b, L)                 # fold; reduce(f, L, init)
```

- 推導式通常比 map/filter 清楚; reduce 加總不如 `sum`; map 適合搭配既有具名函式 (`map(int,...)`)。

15. 遞迴 / Memoization

- 遞迴 = base case + 縮小問題: `n * factorial(n-1)`; naive `fib(n-1)+fib(n-2)` 重複計算 $O(2^n)$。
- CPython 沒有 tail-call optimization; 深度上限約 1000 (`sys.getrecursionlimit()`), 超過 RecursionError → 深度大用迴圈。
- Memo: dict cache 存一次 → $O(n)$; `@lru_cache(maxsize=None)` (引數須 hashable; `fib.cache_info()`; LRU = Least Recently Used); `partial(int, base=2)("1010")=10`。

16. 純函式 / 不可變 / 組合

- Pure: 同輸入必同輸出、無副作用 (不改 global、不 print、不讀時間/random); 易測試、可快取。Immutability: `lst+[x]` 回傳新 list; `lst.append(x)` 改原本; str/tuple 要重建。
- compose: `compose(f,g,h)(x) == f(g(h(x)))`; 實作 `for fn in reversed(funcs): x = fn(x)`。

17. Decorator

- `@deco` ≡ `f = deco(f)`; 本質是 HOF (吃函式、回傳函式)。`@functools.wraps`

(func) 保留原函式 __name__ / __doc__。

- 帶參數要三層 (args → func → call): @repeat(3) = repeat(3)(f)。
- 可疊多個, 由下往上套用 (離 def 最近先): @bold @italic def t() ≡ bold(italic(t)) → hi。

```
import functools, time
def timing(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t = time.perf_counter()
        r = func(*args, **kwargs)       # 呼叫原函式
        print(f"{func.__name__} {time.perf_counter()-t:.4f}s")
        return r                        # 別忘 return
    return wrapper
def repeat(n):                          # decorator factory
    def deco(fn):
        @functools.wraps(fn)
        def wrap(*a, **k):
            for _ in range(n): r = fn(*a, **k)
            return r
        return wrap
    return deco
# HW5-1 memoize (遞迴呼叫也會走 wrapper, 因名稱已被重綁)
def memoize_decorator(func):
    cache = {}
    @functools.wraps(func)
    def wrapper(n):
        if n in cache:
            print(f"Cache hit for fibonacci({n})"); return
cache[n]
        r = func(n); cache[n] = r
        print(f"Computed fibonacci({n}) = {r}")
        return r
    return wrapper
@memoize_decorator
def fibonacci(n): ...   # 題目給定的遞迴版
print("Final Result:", fibonacci(int(input())))
# N=3: Computed 1, Computed 0, Computed 2, Cache hit 1,
Computed 3
```

18. Iterator / Generator

- Iterable 有 __iter__; Iterator 有 __iter__ + __next__; it=iter(L), next(it); 耗盡丟 StopIteration。for = iter() + 反覆 next() 直到 StopIteration。Iterator 只能走一次。
- Generator: 含 yield 的函式; 呼叫時不執行, 回傳 generator (也是 iterator); 每次 next 跑到下個 yield 暫停; 惰性、省記憶體。生成式 (x*x for x in r) (小括號) 約 200 bytes, list 則 MB 級; sum(gen) 串流處理。
- 耗盡: g=(x for x in range(3)); list(g); list(g) → [0,1,2] 然後 []。
- yield from sub 委派; pipeline squared(evens(read(r))) 每階段皆惰性。無窮: while True: yield n, 取用 islice(g,5), takewhile, itertools.count cycle chain。

```
class Counter:                          # 自訂 iterator
    def __init__(s,a,b): s.cur=a; s.end=b
    def __iter__(s): return s
    def __next__(s):
        if s.cur >= s.end: raise StopIteration
        s.cur += 1; return s.cur-1
def fib_gen():                          # 無窮 generator
    a, b = 0, 1
    while True: yield a; a, b = b, a+b
```

Lab5 / HW5 模板

```
# 5-1 偶數和 (*args); 以 END 結束
def even_sum(*args): return sum(x for x in args if x%2==0)
while True:
    line = input()
    if line == "END": break
    print(even_sum(*map(int, line.split())))   # 解包呼叫
# 5-2 3 空罐換 1 瓶, 可借罐子 -> 答案 = N + N//2 (8->12,
9->13)
def cola(n):
    total = e = n
    while e >= 3: k=e//3; total+=k; e=e%3+k
    return total + (1 if e == 2 else 0)        # 剩2罐:借1罐換
1瓶
# HW5-2 G = sum_{i<j<=N} gcd(i,j); 以 0 結束 (先預處理 N<=500)
from math import gcd
S=[0]*501
for n in range(2,501):
    S[n] = S[n-1] + sum(gcd(i,n) for i in range(1,n))
# S[10]=67  S[100]=13015  S[500]=442011
```

19. 檔案 I/O (Lec6)

- 模式: `r` 讀 (不存在 FileNotFoundError); `w` 寫 (建立, 已存在會清空); `a` 附加; `x` 獨佔建立 (已存在 FileExistsError); `b` 二進位; `t` 文字 (預設); `+` 讀寫 (`r+`)。
- `with open(p, "r", encoding="utf-8") as f:` 離開區塊自動 close, 即使發生例外 (≡ try/finally); 離開後 `f.closed` 為 True; 可 `with open(a) as i, open(b,"w") as o:`。不用 with 時必須自己 `close()`。
- 讀: `read()` 全部; `read(n)` n 字元; `readline()` 一行; `readlines()` list (含 `\n`); `for line in f:` 逐行省記憶體 (`line.strip()` 去換行)。
- 寫: `write(s)` 不會自動換行; `writelines(list)` 也不加換行。
- `os: getcwd() chdir() path.exists() isfile() isdir() getsize() join() split() splitext(), os.path.join` 跨平台。`pathlib: p=Path("d")/"in.txt"; exists() is_file() name stem suffix parent read_text() write_text(s) iterdir()`。
- 自訂 context manager: `__enter__` (回傳值給 as) 與 `__exit__(self,*exc)` (一定執行)。

```
import csv, json
with open("a.csv", newline="", encoding="utf-8") as f:    # CSV 要 newline=""
    r = csv.reader(f); header = next(r)        # 每列 list[str]
    for row in r: ...
    for row in csv.DictReader(f): row["name"] # 值皆為 str
w = csv.writer(f); w.writerow([...]); w.writerows(rows)
w = csv.DictWriter(f, fieldnames=[...]); w.writeheader(); w.writerows(ds)
csv.reader(f, delimiter="\t")                  # TSV
obj = json.load(f);  obj = json.loads(s)       # file / str -> dict,list
json.dump(obj, f, indent=4, ensure_ascii=False)  # 中文不轉義
s = json.dumps(obj, indent=2)                  # -> str
# 更新 JSON: load -> 改 -> 用 "w" dump 回去; True->true None->null
```

20. 例外處理

- C 靠回傳碼/errno 每次檢查; Python raise 後沿呼叫堆疊往上找最近的 except, 未處理 → traceback 終止程式。

```
BaseException
 +- SystemExit (sys.exit), KeyboardInterrupt (Ctrl-C)
 +- Exception
     +- ArithmeticError -> ZeroDivisionError
     +- LookupError -> IndexError, KeyError
     +- OSError -> FileNotFoundError, PermissionError
     +- ValueError, TypeError, AttributeError, EOFError
     +- ImportError -> ModuleNotFoundError
```

- 觸發: `10/0` ZeroDivisionError; `int("abc")` ValueError; `"5"+5` TypeError; `open("no")` FileNotFoundError; `[1,2,3][10]` IndexError; `{}["c"]` KeyError; `"s".append()` AttributeError; `import nope` ModuleNotFoundError; 到 EOF 再 `input()` EOFError。

```
try:
    risky()
except (ValueError, ZeroDivisionError) as e:  # tuple 多型別
    print(type(e).__name__, e)
except Exception as e: ...                    # 較廣的放後面
else:      ...   # 沒發生例外才執行 (成功路徑)
finally:   ...   # 一定執行 (清理); try 內有 return 也會先跑
```

- except 由上而下, 第一個符合者勝出: 父類別放前面會遮蔽子類 (`except Exception` 在前 → `except ValueError` 永不執行; `except LookupError` 在前 → KeyError 區塊不執行)。
- bare `except:` 等於 `except BaseException`, 連 Ctrl-C / SystemExit / bug 都吞 → 永遠指定型別; 至少用 `except Exception`。
- `def f(): try: return 1 finally: print("cleanup")` → `print(f())` 先印 cleanup 再印 1。
- EAFP (先做, 失敗才處理; Pythonic; 無 check-use 之間的 race) vs LBYL (先檢查 `os.path.exists` 再 open, 之後檔案可能消失)。
- `raise ValueError("msg")`; 裸 `raise` 重拋同一例外 (記 log 後往上傳); `raise ConfigError("x") from e` 例外鏈 (traceback 同時保留兩個, 顯示 direct cause)。
- 自訂: `class MyErr(Exception): pass`; 加屬性 `__init__(self,msg,field)` + `super().__init__(msg)`; 階層 `DBError` → `QueryError DataNotFoundError` → `except DBError` 捕捉所有子類 (特定子類 except 放前面)。
- `assert cond, "msg"` 失敗丟 AssertionError; 用於內部檢查/前後條件/除錯; 不可用來驗證使用者輸入或檔案是否存在 (`python -O` 會移除 assert) → 用 if/raise 或 try-except。

21. 物件導向 (Lec7)

- 四大支柱: 封裝 Encapsulation, 抽象 Abstraction, 繼承 Inheritance, 多型 Polymorphism。class = 藍圖; object = 實例; `__init__` 為建構子 (建立後自動呼叫, 初始化屬性, 可驗證並 raise)。

```
class Dog:
    species = "Canis"            # class var: 全體共用
    def __init__(self, name):    # self = 呼叫此方法的物件
        self.name = name         # instance var
    def bark(self): return f"{self.name}: Woof"
d = Dog("Rex");  d.bark()        # 等同 Dog.bark(d)
```

- self 是物件本身, 由 Python 自動當第一個引數傳入, 必須明寫 (C++ 隱含 this); 方法漏寫 self → `C().greet()` TypeError (多傳一個引數)。
- class var 全體共用 (`Person.population += 1`); mutable class var (如 list) 所有物件共用 → 在 `__init__` 建 `self.tricks=[]`。`a.items=[1]` 只在 a 上建實例屬性(遮蔽 class var), `b.items` 仍是共用那個; `Counter.count` 以類別名改; `self.count+=1` 會建實例屬性。
- `__del__` 解構子 (物件即將被銷毀; 幾乎用不到, 清理請用 `with`)。
- `__str__` 給使用者 (`print str()`); `__repr__` 給開發者 (REPL/除錯, 要明確); 沒有 `__str__` 時退回 `__repr__`; 都沒有則印記憶體位址。
- `@dataclass` (欄位需型別註記 `x: int`) 自動產生 `__init__ __repr__ __eq__`; 沒定義 `__eq__` 時 `==` 比身分: `P(1)==P(1)` 為 False。dunder: `__eq__ __add__ __len__ __lt__`; 型別不符回 `NotImplemented`; 自訂 `__eq__` 又沒寫 `__hash__` → 不可 hash。

方法	第一參數	存取	用途
instance	`self`	實例 + 類別	一般操作
`@classmethod`	`cls`	類別	替代建構子 `cls(...)`
`@staticmethod`	無	皆不能	工具函式 (可不建物件呼叫)

- 繼承: `class Dog(Animal):`, `super().__init__(name,age)`, override 同名方法, 新增方法; `isinstance(duck, Animal)`。多重繼承 `class Duck(Flyable, Swimmable,Animal)`。
- MRO (方法解析順序, C3): `D.__mro__` → `[D,B,C,A,object]`; `super()` 沿 MRO 走而非只找直接父類; 菱形繼承每個類別只執行一次: `D().who()` 印 `B C A`。
- 多型: override (同介面不同實作 `for s in shapes: s.area()`) + duck typing (只看有沒有該方法, 不檢查型別或繼承)。
- ABC: `class Shape(ABC):` + `@abstractmethod def area(self)`; 子類沒實作就不能實例化 (TypeError); 強制共同介面。
- 封裝: 沒有真正 private; `_x` 慣例內部; `__x` name mangling 成 `_Class__x` (仍可存取)。`@property` 取代冗長 getter/setter:

```
class Person:
    def __init__(self, age): self.age = age    # 走 setter 驗證
    @property
    def age(self): return self._age
    @age.setter
    def age(self, v):
        if not isinstance(v,int): raise TypeError
        if v < 0: raise ValueError("Invalid age")
        self._age = v
    @property
    def is_adult(self): return self._age >= 18  # 唯讀/計算屬性
# p.is_adult = False -> AttributeError (無 setter)
```

- Python vs C++: 無編譯期 private (慣例); 無方法 overloading (用預設值/`*args`); 無 templates (duck typing); 預設皆 public 且動態。

22. 2025 期中考古題: 觀念與填空

- 選擇題 1–12: 1B venv 隔離套件相依; 2C (混用 tab/space 是錯的); 3C 次數不定、條件成立才停; 4C (list comp 不能完全取代 for); 5B tuple 不可變; 6B f-string 比% / format 快; 7C pass by object reference; 8C (decorator 可疊多個); 9B lambda 給 map/filter 當簡單引數; 10B with 自動關檔, 例外也釋放; 11B classmethod 需 cls, staticmethod 不需; 12C duck typing 多型。
- Candy (山峰/山谷一次掃描, 全部先給 1): 上坡累加 1,2,..peak; 下坡累加 1,2,..valley; 山頂重複算要扣 `min(peak,valley)`。(1) `ratings[i] > ratings[i-1]` (2) `i == n` (3) `+=` (4) `min`。
- Jump Game (貪心 maxreach): (5) `n - 1` (6) `i > maxreach` (走不到 i) (7) `maxreach`。
- Sort List (merge sort; 快慢指標找中點, `slow.next=None` 斷開): (8) `self.sortList(head)` (9) `self.sortList(mid)` (10) `self.merge(left, right)`。

```
# Candy 另解: 兩次掃描
c = [1]*n
for i in range(1, n):
    if r[i] > r[i-1]: c[i] = c[i-1] + 1
for i in range(n-2, -1, -1):
    if r[i] > r[i+1]: c[i] = max(c[i], c[i+1] + 1)
ans = sum(c)
```

23. 考古題程式題模板

```
# C1 最短輸入時間: 打字 0.3s/字, 貼上 C 0.4s (整數 0.1s 避免浮點誤差)
T = input(); C = input(); n, m = len(T), len(C)   # 不要 strip
dp = [0] + [10**9]*n
```

```
for i in range(1, n+1):
    dp[i] = dp[i-1] + 3
    if i >= m and T[i-m:i] == C: dp[i] = min(dp[i], dp[i-m] +
4)
print(f"{dp[n]/10:.1f}")                                  # 範例 24.0
# C2 螺旋矩陣: 右->上->左->下, 步長 1,1,2,2,3,3,...
n = int(input()); g = [[0]*n for _ in range(n)]
r = n//2; c = n//2 if n%2 else n//2-1                     # 偶數: 中央
四格左下
v = 1; g[r][c] = 1
D = [(0,1),(-1,0),(0,-1),(1,0)]; d = 0; step = 1
while v < n*n:
    for _ in range(2):
        for _ in range(step):
            if v == n*n: break
            r += D[d][0]; c += D[d][1]; v += 1; g[r][c] = v
        d = (d+1) % 4
    step += 1
w = len(str(n*n))                                          # 右對齊,
行尾無空白
for row in g: print(" ".join(str(x).rjust(w) for x in row))
# C3 鬼屋路徑數: 四方向 -> 簡單路徑 DFS + 回溯 (只能右/下則改
DP)
n = int(input()); g = [list(map(int,input().split())) for _ in
 range(n)]
vis = [[0]*n for _ in range(n)]
def dfs(r, c):
    if (r, c) == (n-1, n-1): return 1
    vis[r][c] = 1; t = 0
    for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
        a, b = r+dr, c+dc
        if 0<=a<n and 0<=b<n and not g[a][b] and not vis[a][b
]:
            t += dfs(a, b)
    vis[r][c] = 0                                          # 回溯
    return t
print(0 if g[0][0] or g[-1][-1] else dfs(0, 0))    # 範例 2
```

24. 快速陷阱表 (Guess the output)

`a=[1,2,3]; b=a; b.append(4); a`	`[1,2,3,4]`
`add(1); add(2)` 預設 `acc=[]`	`[1] [1,2]`
`{1:"a",1.0:"b",True:"c"}`	`{1: 'c'}`
`d=dict.fromkeys("ab",[]); d["a"].append(1)`	`{'a':[1],'b':[1]}`
`[f() for f in [lambda: i for i in range(3)]]`	`[2,2,2]`
`g=(x for x in range(3)); list(g); list(g)`	`[0,1,2] []`
`"https://shop.example.com/".strip("https://")`	`'op.example.com'`
`try: {}['k']  except LookupError: A  except KeyError: B`	印 `A`
`def f(): try: return 1 finally: print("cleanup")`	`cleanup` 再 `1`
`class P: ...; P(1)==P(1)` (無 `__eq__`)	`False`
`class C: def greet(): ...; C().greet()`	`TypeError`
三個 `Counter()` 物件, `Counter.count`	`3, a.count b.count` 皆 3
`a.items=[1]` 後 `b.items` (class var `[]`)	`[]` (共用 list)
`D(B,C)` 菱形 `who()` 各 print 並 `super()`	`B C A`
`-7//2, -7%2; f"{2.5:.0f}", f"{3.5:.0f}"`	`-4, 1; 2, 4`
`type({}), type(()), type((1,))`	dict, tuple, tuple
`bool(""), bool("False"), bool([0])`	F, T, T
`with open("t","w") as fh: ...; fh.closed`	`True`
`"a b c".split()` vs `'a  b'.split(" ")`	無空字串 vs 有 `''`
`int("3.14"), int(" 7 "), int("３")`	ValueError, 7, 3
`sys.getsizeof((x*x for x in range(10**6)))<1000`	`True`